

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – Sorting



NAMA

Antonny Dzaka Fadhillah

NIM

109082500038

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Array, slice, map

B. Guided

1.

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int

    for i = 0; i < len(arr)-1; i++ { // perulangan luar
        idx_min = i

        for j = i + 1; j < len(arr); j++ { //
perulangan dalam
            if arr[j] < arr[idx_min] { // kondisi
                idx_min = j
            }
        }

        if idx_min != i { // swap
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int
    var i int

    for i = 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d :
", i)
        fmt.Scan(&arrAngka[i])
    }

    fmt.Println()
    fmt.Println("=== SEBELUM SORTING ===")

    for i = 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

```

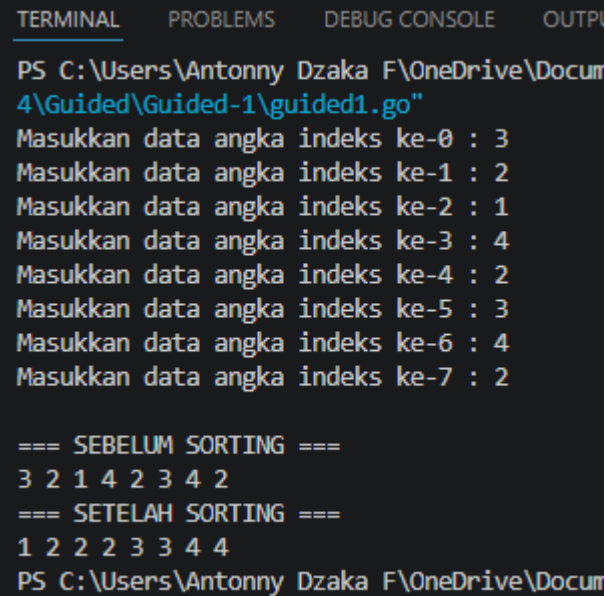
        SelectionSortArray(&arrAngka)

        fmt.Println()
        fmt.Println("=== SETELAH SORTING ===")

        for i = 0; i < len(arrAngka); i++ {
            fmt.Print(arrAngka[i], " ")
        }
    }
}

```

Output :



```

TERMINAL  PROBLEMS  DEBUG CONSOLE  OUTPUT
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\4\Guided\Guided-1\guided1.go"
Masukkan data angka indeks ke-0 : 3
Masukkan data angka indeks ke-1 : 2
Masukkan data angka indeks ke-2 : 1
Masukkan data angka indeks ke-3 : 4
Masukkan data angka indeks ke-4 : 2
Masukkan data angka indeks ke-5 : 3
Masukkan data angka indeks ke-6 : 4
Masukkan data angka indeks ke-7 : 2

=== SEBELUM SORTING ===
3 2 1 4 2 3 4 2
=== SETELAH SORTING ===
1 2 2 2 3 3 4 4
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\4\Guided\Guided-1\guided1.go"

```

2.

```

package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS)-1; i++ {
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ {
            if sMHS[j].IPK < sMHS[idx_min].IPK {

```

```

        idx_min = j
    }
}
if idx_min != i {
    sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
}
}
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n",
i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println(" === SEBELUM SORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println(" === SETELAH SORTING === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\
--- Masukan Data Mahasiswa ke-0 ---
Nama : asdfdfa
NIM : 31241
IPK : 4
--- Masukan Data Mahasiswa ke-1 ---
Nama : afaw
NIM : 322
IPK : 3
--- Masukan Data Mahasiswa ke-2 ---
Nama : afeawfer
NIM : 232
IPK : 3
--- Masukan Data Mahasiswa ke-3 ---
Nama : fgvawrwga
NIM : 23243
IPK : 2
--- Masukan Data Mahasiswa ke-4 ---
Nama : agerwga
NIM : 2345
IPK : 1

=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : asdfdfa
NIM : 31241
IPK : 4.00
--- Data Mahasiswa Ke-1 ---
Nama : afaw
NIM : 322
IPK : 3.00
--- Data Mahasiswa Ke-2 ---
Nama : afeawfer
NIM : 232
IPK : 3.00
--- Data Mahasiswa Ke-3 ---
Nama : fgvawrwga
NIM : 23243
IPK : 2.00
--- Data Mahasiswa Ke-4 ---
Nama : agerwga
NIM : 2345
IPK : 1.00
=====

=== SETELAH SORTING ===
Nama : agerwga
NIM : 2345
IPK : 1.00
Nama : fgvawrwga
NIM : 23243
IPK : 2.00
Nama : afeawfer
NIM : 232
IPK : 3.00
Nama : afaw
NIM : 322
IPK : 3.00
Nama : asdfdfa
NIM : 31241
IPK : 4.00
=====
```

3.

```
package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n-1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting:")
    for i = 0; i < n; i++{
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending):")
    for i =0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}
```

Output :

```
PS C:\Users\Antonny Dzaka F\OneDrive\Desktop\k 12 - modul 14\Guided\Guided-3\gu
Masukkan jumlah data: 2
Masukkan data:
32
21

Data sebelum sorting:
32 21

Data setelah sorting (Descending):
32 21
```

4.

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   string
    ipk   float64
}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j+1] = T[j]
            j = j - 1
        }
        T[j+1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)
```

```

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i+1)

        fmt.Print("Nama : ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM : ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK : ")
        fmt.Scan(&data[i].ipk)
    }

    fmt.Println("\nData sebelum sorting:")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData setelah sorting (Descending
berdasarkan Nama):")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }
}

```

Output :

```

PS C:\Users\Antonny Dzaka F\OneDrive\Documents\ppppp\109082500038_antoony-dzaka-fadhillah> go
4\Guided\Guided-4\guided4.go
Masukkan jumlah mahasiswa: 2

Data mahasiswa ke- 1
Nama : gsedf
NIM : 23
IPK : 3

Data mahasiswa ke- 2
Nama : dfgbzs
NIM : 23
IPK : 2

Data sebelum sorting:
gsedf 23 3
dfgbzs 23 2

Data setelah sorting (Descending berdasarkan Nama):
gsedf 23 3
dfgbzs 23 2
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\ppppp\109082500038_antoony-dzaka-fadhillah>

```


C. Unguided

Soal Unguided 1

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    input(n)
}

func input(n int){
    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)
        arr := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }
        selectionSort(arr)
    }
}

func selectionSort(data []int) {
    for i := 0; i < len(data)-1; i++ {
        min := i
        for j := i + 1; j < len(data); j++ {
            if data[j] < data[min]{
                min = j
            }
        }
        data[i], data[min] = data[min], data[i]
    }
    read(data)
}

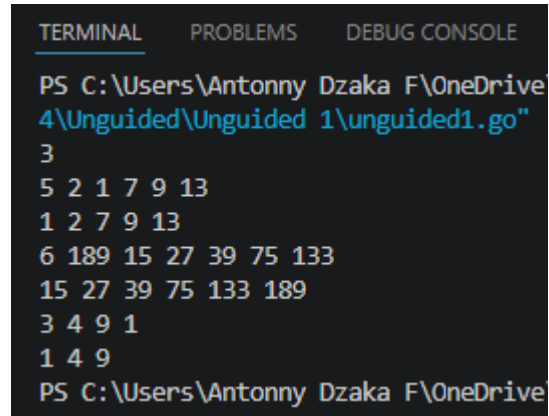
func read(data []int){
```

```

        for _, vel := range data {
            fmt.Print(vel, " ")
        }
        fmt.Println()
    }
}

```

Output :



```

TERMINAL  PROBLEMS  DEBUG CONSOLE
PS C:\Users\Antonny Dzaka F\OneDrive
4\Unguided\Unguided 1\unguided1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Users\Antonny Dzaka F\OneDrive

```

Penjelasan :

total daerah (n) melalui fungsi main dan memprosesnya di fungsi input untuk membaca jumlah rumah (m) beserta nomor-nomornya ke dalam sebuah *slice* pada setiap daerah. *Slice* berisi nomor rumah tersebut kemudian dikirim ke fungsi *selectionSort* untuk diurutkan dari nilai terkecil ke terbesar secara bertahap dengan cara terus mencari nilai minimum di sisa data yang belum terurut dan menukarnya ke posisi paling depan. Setelah proses pengurutan selesai, program langsung mengarahkan data tersebut ke fungsi *read* untuk dicetak ke layar sebagai deretan angka yang dipisahkan oleh spasi.

Soal Unguided 2

```

package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)
    input(n)
}

func input(n int) {
    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)
    }
}

```

```

        arr := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&arr[j])
        }
        selectionSort(arr)
    }
}

func selectionSort(data []int) {
    for i := 0; i < len(data)-1; i++ {
        min := i
        for j := i + 1; j < len(data); j++ {
            if data[j] < data[min] {
                min = j
            }
        }
        data[i], data[min] = data[min], data[i]
    }
    for _, vel := range data {
        if vel%2 != 0 {
            fmt.Print(vel, " ")
        }
    }
    for i := 0; i < len(data)-1; i++ {
        max := i
        for j := i + 1; j < len(data); j++ {
            if data[j] > data[max] {
                max = j
            }
        }
        data[i], data[max] = data[max], data[i]
    }
    for _, vel := range data {
        if vel%2 == 0 {
            fmt.Print(vel, " ")
        }
    }
}

```

Output :

```
TERMINAL  PROBLEMS  DEBUG CONSOLE
PS C:\Users\Antonny Dzaka F\OneDrive\
4\Unguided\Unguided 2\Unguided2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Users\Antonny Dzaka F\OneDrive\
```

Penjelasan :

Program ini menggunakan *Selection Sort* untuk mengurutkan input angka secara membesar (*ascending*) guna mencetak seluruh bilangan ganjil terlebih dahulu, lalu mengurutkannya kembali secara mengecil (*descending*) untuk mencetak seluruh bilangan genapnya, namun tanpa memberikan jeda baris baru antar kelompok data.

Soal Unguided 3

Jawaban :

```
package main

import "fmt"

func main() {
    var arr []int
    var nomer int

    for {
        fmt.Scan(&nomer)
        if nomer < 0 {
            break
        }
        arr = append(arr, nomer)
    }
    Insertion(arr)

    for i := 0; i < len(arr); i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(arr[i])
    }
    fmt.Println()
}
```

```

        if len(arr) < 2 {
            fmt.Println("Data berjarak 0") // Jika data
            kurang dari 2 elemen
        } else {
            diff := arr[1] - arr[0]
            isTetap := true

            for i := 2; i < len(arr); i++ {
                if arr[i]-arr[i-1] != diff {
                    isTetap = false
                    break
                }
            }

            if isTetap {
                fmt.Printf("Data berjarak %d\n", diff)
            } else {
                fmt.Println("Data berjarak tidak tetap")
            }
        }
    }

    func Insertion(arr []int) {
        for i := 1; i < len(arr); i++ {
            key := arr[i]
            j := i - 1
            for j >= 0 && arr[j] > key {
                arr[j+1] = arr[j]
                j--
            }
            arr[j+1] = key
        }
    }

```

Output :

```
TERMINAL  PROBLEMS  DEBUG CONSOLE  OUTPUT  TE
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\p
4\Unguided\Unguided 3\unguided3.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\p
4\Unguided\Unguided 3\unguided3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\p
```

Penjelasan :

Program akan membaca input angka terus-menerus dan menyimpannya ke dalam *slice* selama angka tersebut tidak negatif. Setelah itu, array diurutkan membesar menggunakan *Insertion Sort* (dengan membandingkan tiap elemen ke elemen sebelumnya dan menyisipkannya di posisi yang tepat), lalu dicetak. Terakhir, program menghitung selisih antara elemen pertama dan kedua sebagai patokan jarak (*diff*), lalu memeriksa sisa elemen; jika ada satupun selisih elemen berdekatan yang berbeda dari patokan tersebut, statusnya menjadi "tidak tetap", jika semuanya sama maka dicetak jarak tetapnya.

Soal Unguided 4

Jawaban :

```
package main

import "fmt"

const nMax = 7919

type Buku struct {
    id          string
    judul       string
    penulis     string
    penerbit    string
    eksemplar   int
    tahun       int
    rating      int
}

type DaftarBuku []Buku

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    *pustaka = make(DaftarBuku, n)
```

```

        for i := 0; i < n; i++ {
            fmt.Printf("Masukkan data buku ke-%d\n", i+1)
            fmt.Print("ID: ")
            fmt.Scanln(&(*pustaka)[i].id)
            fmt.Print("Judul: ")
            fmt.Scanln(&(*pustaka)[i].judul)
            fmt.Print("Penulis: ")
            fmt.Scanln(&(*pustaka)[i].penulis)
            fmt.Print("Penerbit: ")
            fmt.Scanln(&(*pustaka)[i].penerbit)
            fmt.Print("Tahun: ")
            fmt.Scanln(&(*pustaka)[i].tahun)
            fmt.Print("Eksemplar: ")
            fmt.Scanln(&(*pustaka)[i].eksemplar)
            fmt.Print("Rating: ")
            fmt.Scanln(&(*pustaka)[i].rating)
            fmt.Println()
        }
    }

func CetakTerfavorit(pustaka DaftarBuku) {
    if len(pustaka) == 0 {
        fmt.Println("Tidak ada buku dalam pustaka.")
        return
    }
    maxRating := pustaka[0].rating
    idx := 0
    for i := 1; i < len(pustaka); i++ {
        if pustaka[i].rating > maxRating {
            maxRating = pustaka[i].rating
            idx = i
        }
    }
    buku := pustaka[idx]
    fmt.Println("Buku terfavorit:")
    fmt.Printf("Judul: %s\n", buku.judul)
    fmt.Printf("Penulis: %s\n", buku.penulis)
    fmt.Printf("Penerbit: %s\n", buku.penerbit)
    fmt.Printf("Tahun: %d\n", buku.tahun)
}

func UrutBuku(pustaka DaftarBuku) {
    for i := 1; i < len(pustaka); i++ {
        key := pustaka[i]
        j := i - 1
        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

```

```

    }

    func Cetak5Terbaru(pustaka DaftarBuku) {
        jumlah := 5
        if len(pustaka) < 5 {
            jumlah = len(pustaka)
        }
        fmt.Printf("%d buku dengan rating tertinggi:\n",
jumlah)
        for i := 0; i < jumlah; i++ {
            fmt.Printf("%d. %s (Rating: %d)\n", i+1,
pustaka[i].judul, pustaka[i].rating)
        }
    }

    func CariBuku(pustaka DaftarBuku, r int) {
        low := 0
        high := len(pustaka) - 1
        for low <= high {
            mid := (low + high) / 2
            if pustaka[mid].rating == r {
                buku := pustaka[mid]
                fmt.Println("Buku ditemukan:")
                fmt.Printf("Judul: %s\n", buku.judul)
                fmt.Printf("Penulis: %s\n", buku.penulis)
                fmt.Printf("Penerbit: %s\n", buku.penerbit)
                fmt.Printf("Tahun: %d\n", buku.tahun)
                fmt.Printf("Eksemplar: %d\n",
buku.eksemplar)
                fmt.Printf("Rating: %d\n", buku.rating)
                return
            }
            if pustaka[mid].rating > r {
                low = mid + 1
            } else {
                high = mid - 1
            }
        }
        fmt.Println("Tidak ada buku dengan rating seperti
itu.")
    }

    func main() {
        var n int
        fmt.Print("Masukkan jumlah buku: ")
        fmt.Scanln(&n)
        if n <= 0 || n > nMax {
            fmt.Println("Jumlah buku tidak valid.")
            return
        }
        var pustaka DaftarBuku
    }

```



```
    DaftarkanBuku(&pustaka, n)
    CetakTerfavorit(pustaka)
    UrutBuku(pustaka)
    Cetak5Terbaru(pustaka)
    var ratingCari int
    fmt.Print("Masukkan rating buku yang dicari: ")
    fmt.Scanln(&ratingCari)
    CariBuku(pustaka, ratingCari)
}
```

Output :

```

TERMINAL  PROBLEMS  DEBUG CONSOLE  OUTPUT  TEST
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\ppp\4\Unguided\Unguided 4\Unguided4.go"
Masukkan jumlah buku: 2
Masukkan data buku ke-1
ID: 2
Judul: fvadeg
Penulis: gawrs
Penerbit: afgvbeswg
Tahun: 2113
Eksemplar: 3113
Rating: 4

Masukkan data buku ke-2
ID: 4232
Judul: efawfg
Penulis: asdgvgas
Penerbit: regaws
Tahun: 325
Eksemplar: 342
Rating: 3

Buku terfavorit:
Judul: fvadeg
Penulis: gawrs
Penerbit: afgvbeswg
Tahun: 2113
2 buku dengan rating tertinggi:
1. fvadeg (Rating: 4)
2. efawfg (Rating: 3)
Masukkan rating buku yang dicari: 4
Buku ditemukan:
Judul: fvadeg
Penulis: gawrs
Penerbit: afgvbeswg
Tahun: 2113
Eksemplar: 3113
Rating: 4
PS C:\Users\Antonny Dzaka F\OneDrive\Documents\ppp\4\Unguided\Unguided 4\Unguided4.go"

```

Penjelasan :

Program Golang ini merupakan sistem manajemen data pustaka yang bekerja dengan membaca sejumlah detail buku ke dalam sebuah *slice struct*, lalu secara berurutan mencari buku dengan rating tertinggi secara sekuensial, mengurutkan seluruh koleksi buku berdasarkan rating dari terbesar ke terkecil (*descending*) menggunakan algoritma *Insertion Sort* agar dapat mencetak daftar hingga lima buku terfavorit, dan diakhiri dengan fitur pencarian detail

buku berdasarkan nilai rating spesifik menggunakan algoritma *Binary Search* pada kumpulan data yang telah terurut tersebut.

D. Kesimpulan

Selection Sort dan Insertion Sort adalah algoritma pengurutan dasar dengan pendekatan berbeda yang telah kita terapkan dalam bahasa pemrograman Go; *Selection Sort* mengurutkan data dengan cara berulang kali mencari nilai ekstrem (terkecil/terbesar) dari sisa data yang belum terurut lalu menukarnya ke posisi paling depan (seperti pada kasus pengurutan nomor rumah dan pemisahan ganjil-genap), sedangkan *Insertion Sort* bekerja dengan mengambil elemen satu per satu lalu menyisipkannya ke posisi yang tepat di dalam kumpulan data yang sudah terurut sebelumnya (seperti pada kasus pengecekan jarak angka dan pengurutan rating buku). Kedua algoritma ini terbukti sangat penting dan efektif digunakan untuk menyusun data array atau *slice* agar lebih mudah diproses ke tahap selanjutnya, seperti untuk evaluasi pola angka maupun pencarian data spesifik.